The Unofficial Reverse Engineer's Guide to STACKL.

This document will continue to evolve as we piece together this tool.
Things with ?s still need answers. Educated guesses might be 100% off and need to be re-evaluated as we reverse engineer.
Stack layout now appears to be top down:

[1] evaluated left node
[2] evaluated right node.
[0] operation

But there seems to be seems to be some inconsistencies in left-right and right-left based on the operation being performed.
Again – we don't have to be perfect *yet*. But, we need to be consistently updating this document as we progress.

Revision History:
0.1 – Look at the opcode table. Deduce possible usages/outcomes for what the opcode does. Generate boilerplate documentation that can be refined as further reverse engineering work continues. *(Slides 1-27)*
0.2 – Added general syntax. Updated TODO list. Resolved some operation ambiguities from Rev0.1. *(Slides 27-46)*
0.3 - Updated TODO list. Added print/prints library functions. Added temporary explanations for if/while/for loops (keep or delete?). *(Slides 47-71)*

Terminology

Documentation Syntax:

TODO: Apply this to the documentation
Stack defaults to the application memory stack.
STACKL stacks for evaluation (versus) should be explicitly stated.

Stacks:

Data = Mutable. The runtime data stack. Each scope has its own sub-stack that is managed by the application.

Instruction = Non-mutable. The list of stack operations to perform. Generated from the compiler's IL. These are referenced via labels. The iteration of the instruction stack is handled by the IP. When the IP hits an opcode, the data stack is evaluated to perform the underlying operation.

Registers:

BP = Frame Pointer/Base Pointer. Points to a fixed, unchanging location within the current function's stack frame. This allows the program to reliably access function arguments and local variables using constant memory offsets, even as the stack pointer changes during function execution.
(TODO: Deduce how the BP and the data stack change when invoking CALL and RETURN/RETURNV)
FP = Frame Pointer/Base Pointer (TODO: Consolidate so we don't have two interchangeable acronyms)
IP = Instruction Pointer – Instruction stack iterator
SP = Stack Pointer – Data stack iterator

General Syntax

Functions and Procedures:

Functions and procedures are declared in the syntax:

```
.function functionName
```

There is no difference between a function (return value) and procedure (no return value) other than the .function block calling RETURNV (function) or RETURN (procedure).

Functions/procedures are invoked using the CALL operator with the syntax:

```
call @functionName
```

The IP has no direct accessors and can only be adjusted by CALL, JUMP, JUMPE, RETURN, and RETURNV opcodes. STACKL effectively has an instruction cache that plays back the STACKL operations, with the IP referencing the active operation to perform. The underlying opcode performs the commit operation based on the parameters located on the stack as designated by the SP. Some operations, such as CALL, will explicitly modify the stack. Others, such as binary operations, will presume the parameters are predetermined by the value addressed at SP.

TODOs:

Consolidate terminology. Use IL or IR but not both. Use BP or FP but not both. We're at the draft stage. We need to know these mean the same, but end users don't. And mixing terminology – especially in acronym form – makes documentation hard to read.

Operations that push left-right versus right-left need to be clearly identified.

From what we have deduced is that:

* We are using the caller boy scout model. Win! Caller needs to explicitly set up the stack prior to function invocation, then clean up the stack after returning.

    PUSH (parameters)
    CALL (function name)
    POPARGS (parameters)

  In order to avoid memory leaks, we know that we need to cache the size of the parameter list. The PUSH/POPARGS can be skipped if no parameters are present.

From what we have kind of deduced (but not 100%) is that:

* The SP is pointing to the return value when RETURNV is called. SP points to byte 0, which represents either a uint8_t value, or byte 0 of a 4-byte stack value for a uint32_t value.

What we're still unclear of:

* How the frame pointer works. Does CALL inherently update its value and RETURN/RETURNV reset it.

Major concerns

* We don't have either explicit GETFP/SETFP or PUSHFP/POPFP operations. Having those would allow us to ensure we could link-list the FP. It would also allow us to ensure calling convention contracts. As of now, we are the mercy of what CALL and RETURN/RETURNV do.

What we know we can fall back on:

* Arrays have a fixed syntax with fixed sizes. Static access of const intval indexes can be evaluated pre-STACKL during the optimization stages. For using a variable as the index into the array, consider writing a helper function for something like:

 stack_offset = arrayEmit (base_node, index_node)

Then have that helper function wrap the "base_node_offset + (sizeof(base_node_type) * evaluate(index))" functionality.

Burn After Reading:

Handling an if statement

```
if(x == 0)
{
    function1();
}
else
{
    function2();
}
```

```
IF: intermediate_value = (x == 0) // 1 if true, 0 if false
JUMPE ELSE // top of the stack = 0? jump to ELSE
CALL function1
JUMP END // skip the else block
ELSE:
CALL FUNCTION2:
// fall through
END:
```

Handling a while loop

```
while(X == 5)
{
    // code block 1
}
// code block 2
```

```
While: push (X == 5)// 1 if true, 0 if false
JUMPE End // top of the stack = 0? jump to END
code block 1

End:
code block 2
```

Handling a for loop

```
for(int X = 5; X != 0; X--)
{
    // code block 1
}
// code block 2
```

```
For: int X = 5
Jump Eval

Adjustment: X--
Fall through

Eval: if(x == 0) jump End
code block 1
Jump Adjustment

End:
code block 2
```

Opcodes/Stack Operations

ADJSP

Adjust the stack pointer. Used to preallocate on the stack memory for a new stack frame.

STACKL Stack layout:

uint32_t? to increment the stack pointer by.
TODO: Can this be negative? Just because our labs are limited to uint32_t and uint8_t doesn't mean the STACKL code has to be
ADJSP

Registers Affected:
SP

POPARGS

`POPARGS` removes the specified number of bytes and leaves the return value on the top of stack.

Helper operation that caches the return value at SP, decrements SP by the value specified (should match ADJSP/frame size), insert the cached return value at the top of the stack (uint32_t limited).

STACKL Stack layout:
Size of the function/procedure's frame
POPARGS

Registers Affected:
SP

CALL

Invoke a function call. Instructions are labeled. This pushes the IP and current FP to the stack.

STACKL Stack layout:

Label
CALL

Registers Affected:
FP
IP

RETURN

Return from a CALL operation without returning a value. SP must be set to the value it was at the time of entering the CALL procedure. The FP and IP are then restored.

STACKL Stack layout:

RETURN

Registers Affected:

IP
FP?

RETURNV

Return from a CALL operation while returning a value. SP must be set to the value it was at the time of entering the CALL procedure. POPARGS should be used to prep the return value. The FP and IP are then restored, and the return value is on the top of the stack.

STACKL Stack layout:

RETURNV

Registers Affected:

IP
FP

JUMP

Fancy word for those “goto” operations you have been taught not to use. Set the IP to new location with no stack cleanup performed.

STACKL Stack layout:

Label
JUMP

Registers Affected:
IP

JUMPE

Compare VALUE to zero. If the value is zero, call JUMP to the location specified by Label.
TODO: Value = uint32_t or just lower uint8_t?

STACKL Stack layout:

Value
Label
JUMPE

Registers Affected:
IP

POP

Decrement the stack pointer. Since we have no registers, and we have POPVAR/POPVARIND, we can deduce that we are probably going to discard the value entirely? And since no reference has been mentioned yet about a size, we're going to presume it's just decrementing SP by sizeof(uint32_t).

STACKL Stack layout:

POP

Registers Affected:

SP

PUSH

Add an element to the stack.

STACKL Stack layout:

uint32_t Element
PUSH

Registers Affected:
SP

POPCVAR

Pops an uint8_t value off of the stack and stores the value at FP+[Destination].

STACKL Stack layout:

Destination
POPCVAR

Registers Affected:
SP

POPCVARIND

Take the value currently referenced by the SP, use it to reference/resolve a value (BP + offset), populate the destination with the single 1-byte character being referenced, then decrement the SP.

STACKL Stack layout:

Destination?
Offset?
POPCVARIND

Registers Affected:
SP

POPVAR

Pop the top value off of the stack. Assign the Frame Pointer + Destination Offset stack value to the popped value. The resulting stack pointer should remove the popped value, Frame Pointer + Destination Offset value, and POPVAR instruction.

STACKL Stack layout:

Frame Pointer + Destination Offset
POPVAR

Registers Affected:
SP

POPVARIND

Take the offset on the top of the stack to use as the assignment value. Take the next value off of the stack to use as an offset from the frame pointer to store the value. Assign the value in the cache at FP[offset] to the previous top of the stack value.
Decrement the stack pointer by the three 4-byte parameters?

STACKL Stack layout:

Resulting Destination Value
FP+Destination Offset
POPVARIND

Registers Affected:
SP

PUSHCVAR

Take the value at FP+Reference, then push the resulting uint8_t to the stack. The push increments the SP by 1.

STACKL Stack layout:

FP+Offset Reference
PUSHCVAR

Registers Affected:
SP

PUSHCVARIND

Resolve the stack value for BP[source], then push the uint8_t value on the top of the stack to that location.

STACKL Stack layout:

source
PUSHCVARIND

Registers Affected:
SP

PUSHVAR

Take the value at FP+Reference, then push the resulting uint32_t to the stack. The push increments the SP by 4.

STACKL Stack layout:

FP+Offset Reference
PUSHVAR

Registers Affected:
SP

PUSHVARIND

Resolve the stack value for BP[source], then push that uint32_t value to the stack. Increment SP by 4.

STACKL Stack layout:

Source BP Offset
PUSHVARIND

Registers Affected:
SP

PUSHFP

Push the current value of the FP to the stack. Increment SP by 4.

STACKL Stack layout:

source?
PUSHFP

Registers Affected:
SP

AND

Perform a logical AND operation. Pop the left and right operations off of the stack, and then push the result of the operation in its place?

STACKL Stack layout:

left?
right?
AND

Registers Affected:
SP?

DIVIDE

Perform a mathematical divide operation. Pop the left and right operations off of the stack, and then push the result of the operation in its place?

STACKL Stack layout:

left?
right?
DIVIDE

Registers Affected:
SP?

EQ

Perform a binary equation operation. Pop the left and right operations off of the stack, and then push the value 1 (representing true) or 0 (representing false) based on the result of the equality operation?

STACKL Stack layout:

left?
right?
EQ

Registers Affected:
SP?

MINUS

Perform a mathematical subtraction operation. Pop the left and right operations off of the stack, and then push the mathematical operation result onto the stack? (Why use ADD and MINUS? Seems like PLUS/MINUS or ADD/SUBTRACT pair better?)

STACKL Stack layout:

left?
right?
MINUS

Registers Affected:
SP?

MOD

Perform a mathematical modulis operation. Pop the left and right operations off of the stack, and then push the mathematical operation result onto the stack?

STACKL Stack layout:

left?
right?
MOD

Registers Affected:
SP?

NE

Perform a binary comparison on the left and right operators. Pop the left and right operations off of the stack, perform a binary comparison, then push onto the stack 0 (false) if the values are equal, or 1 (true) if the values are not equal?

STACKL Stack layout:

left?
right?
NE

Registers Affected:
SP?

OR

Perform a logical or operation. Pop the left and right operations off of the stack, and then push the logical operation result onto the stack?

STACKL Stack layout:

left?
right?
OR

Registers Affected:
SP?

PLUS

Perform a mathematical addition operation. Pop the left and right operations off of the stack, and then push the mathematical operation result onto the stack?

STACKL Stack layout:

left?
right?
PLUS

Registers Affected:
SP?

TIMES

Perform a mathematical multiplication operation. Pop the left and right operations off of the stack, and then push the mathematical operation result onto the stack? (Why TIMES not MUL?)
Random for the 8-bit developer engineers - if the platform doesn't support MUL, you can use a count register to perform a loop of adds. But you knew that from grade school.

STACKL Stack layout:

left?
right?
MINUS

Registers Affected:
SP?

Library Functions

print

Prints a uint32_t from the top of the stack to the console

Requires:

uint32_t – uint32_t value to print to the console

Returns:

uint32_t? – Resulting success (non-zero) or fail (zero) for the print operation.

Usage:

```
PUSH 5
CALL @print
POP                 Pop the return value from print
POP                 Pop the argument
```

prints

Prints a null terminated character array to the console.

From the slides, we have no idea how this works currently. Our language only supports uint32_t and uint8_t. The intuitive way to use this would be to push the string uint8_t by uint8_t, ensuring a final uint8_t value of 0 to indicate termination, then CALL @prints. But the slides specify isolated .dataseg (data segments) and .codeseg (code segments), which we did not cover in any lecture.

```
.dataseg

$LABEL:

.string "Hello world!\n"

.codeseg

PUSH @LABEL

OUTS
```

So, this might not even be relevant for our lab. But we can easily fake prints by doing:

```
prints("AI/DR");
```

Helper function:

```
void prints(const char* str)
{
   while(*str)
   {
      emit:
        PUSHC *str
        @CALL PRINT
        POP
        POP
      code:
        ++str;
    }
}
```

Obviously, this approach is far less efficient. But it works. And it's something we can fall back on.